



UNIVERSIDAD AUTÓNOMA DE SAN LUIS POTOSÍ

FACULTAD DE INGENIERÍA

AREA MECÁNICA ELÉCTRICA



PROYECTO PARCIAL 1

TRATAMIENTO DE IMÁGENES

ALUMNO: RAMÍREZ GALVÁN MANUEL

CLAVE: 288093

PROFESOR: JULLY KADO MERCADO ELIAS

FECHA: 17/FEBRERO/2025

Contenido

Introducción	3
Desarrollo	3
Explicación del Código para Quitar el Fondo	3
Explicación del Código para unir las Imágenes	4
Código Completo	7
Imágenes Utilizadas	9
Resultados	10
Referencias	12

Ilustración 1	3
Ilustración 2	3
Ilustración 3	3
Ilustración 4	4
Ilustración 5	4
Ilustración 6	4
Ilustración 7	4
Ilustración 8	4
Ilustración 9	4
Ilustración 10	5
Ilustración 11	5
Ilustración 12	5
Ilustración 13	5
Ilustración 14	6
Ilustración 15	6
Ilustración 16	6
Ilustración 17	6
Ilustración 18	6
Ilustración 19	6
Ilustración 20	7
Ilustración 21 Código para quitar fondo	7
Ilustración 22 Código para sobreponer imágenes	8
Ilustración 23 Fotos utilizadas	9
Ilustración 24 Resultado de imágenes sin fondo	10
Ilustración 25 Resultados finales	11

Introducción

Este proyecto busca el uso de Python y OpenCV para eliminar el fondo de imágenes y sobreponerlas todas juntas en un nuevo fondo.

El objetivo es eliminar el fondo de varias imágenes utilizando segmentación en el espacio de color HSV, aprovechando la detección de colores, por ejemplo el color verde, esto para generar una máscara que permite conservar solo a la persona, que es la parte necesaria para después ajustar el tamaño y la posición para agregar cada una de ellas en fondo fijo, esto manteniendo una coherencia entre fotos y fondo.

Para lograr esto se tiene que tener en cuenta, sobre todo, una buena captura de imagen con muy buena iluminación y de preferencia con un solo color de fondo para facilitar el proceso de quitar el fondo y poder hacer lo necesario para encajarla en el fondo.

Desarrollo

Explicación del Código para Quitar el Fondo

Se coloca la librería de OenCV y NumPy además de cargar la imagen original la persona.

```
1 import cv2
2 import numpy as np
3
4 # Cargar la imagen
5 imagen = cv2.imread("imagen9.jpg")
```

Ilustración 1

Se convierte la imagen de RGB a HSV porque los colores son más fáciles de distinguir ya que están separados en Tono (0° a 180°), Saturación (0 a 255) y Brillo (0 a 255) y es mas fácil de detectar el color de fondo.

```
6
7 # Convertir la imagen a HSV
8 hsv = cv2.cvtColor(imagen, cv2.COLOR_BGR2HSV)
```

Ilustración 2

Se definen los limites del color verde en HSV, Tono (35° a 85°), Saturación (40 a 25), Brillo (40 a 255).

```
9
10 # Definir los rangos de color para eliminar el fondo (ajustar según necesidad)
11 lower = np.array([35, 40, 40], dtype=np.uint8) # Rango inferior
12 upper = np.array([85, 255, 255], dtype=np.uint8) # Rango superior
```

Ilustración 3

Crea una máscara binaria donde el fondo se hace blanco (255) y todo lo demás en negro (0) y se invierte la máscara para tener el fondo en negro y todo lo demás en blanco.

```

14 # Crear la máscara
15 mascara = cv2.inRange(hsv, lower, upper)
16
17 # Invertir la máscara para seleccionarme
18 mascara_invertida = cv2.bitwise_not(mascara)

```

Ilustración 4

Se convierte la imagen de 3 canales (RGB) a 5 canales (RGBA), ya que se necesita el canal Alfa (A) para la transparencia.

```

20 # Convertir imagen a 4 canales (RGBA)
21 imagen_rgba = cv2.cvtColor(imagen, cv2.COLOR_BGR2BGRA)

```

Ilustración 5

Se le asigna la máscara invertida al canal Alfa, donde todo lo blanco se vuelve visible y el fondo que es negro, se vuelve transparente.

```

23 # Asignar la máscara como canal alfa (transparencia)
24 imagen_rgba[:, :, 3] = mascara_invertida

```

Ilustración 6

Se guarda la imagen de la persona sin fondo en formato PNG, ya que este soporta las transparencias.

```

26 # Guardar la imagen con transparencia
27 cv2.imwrite("imagen9_sin_fondo.png", imagen_rgba)

```

Ilustración 7

Explicación del Código para unir las Imágenes

Se coloca la librería de OenCV y además de cargar la imagen de para el fondo a color.

```

1 import cv2
2
3 # Cargar la imagen de fondo
4 fondo = cv2.imread("F1.jpg", cv2.IMREAD_COLOR)

```

Ilustración 8

Se redimensiona el fondo para que no ocupe mas tamaño que el tamaño de la pantalla, esto tanto como en alto y en ancho.

Con cv2.resize() y la interpolación cv2.INTER_ARES se reduce la imagen manteniendo la calidad.

```

5
6 # Redimensionar el fondo para que no sea demasiado grande
7 escala_fondo = 0.45 # Ajusta este valor
8 nuevo_ancho_fondo = int(fondo.shape[1] * escala_fondo)
9 nuevo_alto_fondo = int(fondo.shape[0] * escala_fondo)
10 fondo = cv2.resize(fondo, (nuevo_ancho_fondo, nuevo_alto_fondo), interpolation=cv2.INTER_AREA)

```

Ilustración 9

Se crea una lista para agregar cada una de las imágenes de la persona sin fondo, con su posición deseada para que se agregue al fondo, en “X” y en “Y”, además de una escala para cambiar el tamaño de cada una de manera individual.

```
12 # Lista de las 9 imágenes sin fondo con sus posiciones
13 imagenes_a_agregar = [
14     {"archivo": "imagen1_sin_fondo.png", "x": 230, "y": 950, "escala": 0.3},
15     {"archivo": "imagen2_sin_fondo.png", "x": 800, "y": 1100, "escala": 0.3},
16     {"archivo": "imagen3_sin_fondo.png", "x": 450, "y": 900, "escala": 0.2},
17     {"archivo": "imagen4_sin_fondo.png", "x": 960, "y": 940, "escala": 0.1},
18     {"archivo": "imagen5_sin_fondo.png", "x": 0, "y": 1000, "escala": 0.35},
19     {"archivo": "imagen6_sin_fondo.png", "x": 80, "y": 780, "escala": 0.06},
20     {"archivo": "imagen7_sin_fondo.png", "x": 570, "y": 950, "escala": 0.3},
21     {"archivo": "imagen8_sin_fondo.png", "x": 830, "y": 940, "escala": 0.1},
22     {"archivo": "imagen9_sin_fondo.png", "x": 30, "y": 900, "escala": 0.1}
23 ]
```

Ilustración 10

Se ajustan las posiciones de las imágenes al nuevo tamaño del fondo, ya que el fondo puede cambiar su tamaño y se deben de ajustar las posiciones de todas las imágenes, esto se hace multiplicando las posiciones “X” y “Y” por la escala del fondo.

```
25 # Ajustar las posiciones de las imágenes al nuevo tamaño del fondo
26 for img in imagenes_a_agregar:
27     img["x"] = int(img["x"] * escala_fondo)
28     img["y"] = int(img["y"] * escala_fondo)
```

Ilustración 11

Se utiliza un ciclo for para colocar cada una de las imágenes en el fondo, donde se carga cada imagen con el canal Alfa.

```
30 # Recorrer la lista de imágenes y agregarlas al fondo
31 for img_info in imagenes_a_agregar:
32     # Cargar la imagen sin fondo (con transparencia)
33     imagen_transparente = cv2.imread(img_info["archivo"], cv2.IMREAD_UNCHANGED)
34
```

Ilustración 12

Se obtienen las dimensiones de la imagen sin fondo para ajustar el tamaño de la imagen para que quede mejor en el fondo ya escalado. La escala de cada imagen se ajusta proporcionalmente adicionando que se vean bien las imágenes.

Después, obtener las nuevas dimensiones de las imágenes después del escalado y definir en donde se colocará la imagen.

```
35 # Obtener dimensiones de la imagen sin fondo
36 h_img, w_img, _ = imagen_transparente.shape
37
38 # Escalar la imagen según el tamaño del fondo
39 escala = img_info["escala"] * escala_fondo
40 nuevo_ancho = int(w_img * escala)
41 nuevo_alto = int(h_img * escala)
42 imagen_transparente = cv2.resize(imagen_transparente, (nuevo_ancho, nuevo_alto), interpolation=cv2.INTER_AREA)
43
44 # Obtener nuevas dimensiones después del escalado
45 h_img, w_img, _ = imagen_transparente.shape
46
47 # Definir la posición donde se colocará la imagen
48 x_offset, y_offset = img_info["x"], img_info["y"]
```

Ilustración 13

Verifica si la imagen cabe dentro del fondo, si la imagen es mas grande que el fondo, se ajustara para que quepa, evitando que las imágenes sean cortadas o se queden fuera del fondo.

```
50 # Verificar si la imagen cabe dentro del fondo
51 h_fondo, w_fondo, _ = fondo.shape
52 if y_offset + h_img > h_fondo or x_offset + w_img > w_fondo:
53     print(f"La imagen {img_info['archivo']} no cabe en el fondo, ajustando tamaño...")
54     escala = min((w_fondo - x_offset) / w_img, (h_fondo - y_offset) / h_img) * 0.5
55     nuevo_ancho = int(w_img * escala)
56     nuevo_alto = int(h_img * escala)
57     imagen_transparente = cv2.resize(imagen_transparente, (nuevo_ancho, nuevo_alto), interpolation=cv2.INTER_AREA)
```

Ilustración 14

Se extraen los canales RGB y Alfa para tenerlos por separados y usar posteriormente el canal Alfa.

```
59 # Extraer los canales BGR y Alfa
60 b, g, r, a = cv2.split(imagen_transparente)
```

Ilustración 15

Convierte la máscara Alfa con el canal Alfa a valores entre 0 y 1 para aplicar la transparencia de manera correcta.

```
62 # Normalizar la máscara alfa
63 mascara_alpha = a / 255.0
```

Ilustración 16

Extrae y recorta la parte del fondo donde se colocará la imagen.

```
65 # Recortar la región del fondo donde se colocará la imagen
66 fondo_recorte = fondo[y_offset:y_offset+h_img, x_offset:x_offset+w_img]
```

Ilustración 17

Fusiona la imagen de la persona sin fondo sobre el fondo usando la máscara de transparencia.

```
68 # Aplicar la imagen sin fondo sobre el fondo
69 for c in range(3): # BGR
70     fondo_recorte[:, :, c] = (fondo_recorte[:, :, c] * (1 - mascara_alpha) +
71     imagen_transparente[:, :, c] * mascara_alpha)
```

Ilustración 18

Inserta la imagen sin fondo en la posición correspondiente del fondo.

```
73 # Insertar la imagen combinada en el fondo original
74 fondo[y_offset:y_offset+h_img, x_offset:x_offset+w_img] = fondo_recorte
```

Ilustración 19

Guarda la imagen final con formato PNG y muestra una ventana con la imagen hasta que se realice una acción para cerrarla.

```
76 # Guardar y mostrar el resultado
77 cv2.imwrite("imagen_final1.png", fondo)
78 cv2.imshow("Imagen Final", fondo)
79 cv2.waitKey(0)
80 cv2.destroyAllWindows()
```

Ilustración 20

Código Completo

Este código se repitió para cada una de las nueve imágenes.

```
import cv2
import numpy as np

# Cargar la imagen
imagen = cv2.imread("imagen9.jpg")

# Convertir la imagen a HSV
hsv = cv2.cvtColor(imagen, cv2.COLOR_BGR2HSV)

# Definir los rangos de color para eliminar el fondo (ajustar según necesidad)
lower = np.array([35, 40, 40], dtype=np.uint8) # Rango inferior
upper = np.array([85, 255, 255], dtype=np.uint8) # Rango superior

# Crear la máscara
mascara = cv2.inRange(hsv, lower, upper)

# Invertir la máscara para seleccionarme
mascara_invertida = cv2.bitwise_not(mascara)

# Convertir imagen a 4 canales (RGBA)
imagen_rgba = cv2.cvtColor(imagen, cv2.COLOR_BGR2BGRA)

# Asignar la máscara como canal alfa (transparencia)
imagen_rgba[:, :, 3] = mascara_invertida

# Guardar la imagen con transparencia
cv2.imwrite("imagen9_sin_fondo.png", imagen_rgba)
```

Ilustración 21 Código para quitar fondo

Se hicieron 3 códigos solo para cambiar las posiciones, escalas de las imágenes y el fondo.

```
import cv2

# Cargar la imagen de fondo
fondo = cv2.imread("F1.jpg", cv2.IMREAD_COLOR)

# Redimensionar el fondo para que no sea demasiado grande
escala_fondo = 0.45 # Ajusta este valor
nuevo_ancho_fondo = int(fondo.shape[1] * escala_fondo)
nuevo_alto_fondo = int(fondo.shape[0] * escala_fondo)
fondo = cv2.resize(fondo, (nuevo_ancho_fondo, nuevo_alto_fondo), interpolation=cv2.INTER_AREA)

# Lista de las 9 imágenes sin fondo con sus posiciones
imagenes_a_agregar = [
    {"archivo": "imagen1_sin_fondo.png", "x": 230, "y": 950, "escala": 0.3},
    {"archivo": "imagen2_sin_fondo.png", "x": 800, "y": 1100, "escala": 0.3},
    {"archivo": "imagen3_sin_fondo.png", "x": 450, "y": 900, "escala": 0.2},
    {"archivo": "imagen4_sin_fondo.png", "x": 960, "y": 940, "escala": 0.1},
    {"archivo": "imagen5_sin_fondo.png", "x": 0, "y": 1000, "escala": 0.35},
    {"archivo": "imagen6_sin_fondo.png", "x": 80, "y": 780, "escala": 0.06},
    {"archivo": "imagen7_sin_fondo.png", "x": 570, "y": 950, "escala": 0.3},
    {"archivo": "imagen8_sin_fondo.png", "x": 830, "y": 940, "escala": 0.1},
    {"archivo": "imagen9_sin_fondo.png", "x": 30, "y": 900, "escala": 0.1}
]

# Ajustar las posiciones de las imágenes al nuevo tamaño del fondo
for img in imagenes_a_agregar:
    img["x"] = int(img["x"] * escala_fondo)
    img["y"] = int(img["y"] * escala_fondo)

# Recorrer la lista de imágenes y agregarlas al fondo
for img_info in imagenes_a_agregar:
    # Cargar la imagen sin fondo (con transparencia)
    imagen_transparente = cv2.imread(img_info["archivo"], cv2.IMREAD_UNCHANGED)

    # Obtener dimensiones de la imagen sin fondo
    h_img, w_img, _ = imagen_transparente.shape

    # Escalar la imagen según el tamaño del fondo
    escala = img_info["escala"] * escala_fondo
    nuevo_ancho = int(w_img * escala)
    nuevo_alto = int(h_img * escala)
    imagen_transparente = cv2.resize(imagen_transparente, (nuevo_ancho, nuevo_alto),
    interpolation=cv2.INTER_AREA)

    # Obtener nuevas dimensiones después del escalado
    h_img, w_img, _ = imagen_transparente.shape

    # Definir la posición donde se colocará la imagen
    x_offset, y_offset = img_info["x"], img_info["y"]

    # Verificar si la imagen cabe dentro del fondo
    h_fondo, w_fondo, _ = fondo.shape
    if y_offset + h_img > h_fondo or x_offset + w_img > w_fondo:
        print(f"La imagen {img_info['archivo']} no cabe en el fondo, ajustando tamaño...")
        escala = min((w_fondo - x_offset) / w_img, (h_fondo - y_offset) / h_img) * 0.5
        nuevo_ancho = int(w_img * escala)
        nuevo_alto = int(h_img * escala)
        imagen_transparente = cv2.resize(imagen_transparente, (nuevo_ancho, nuevo_alto),
        interpolation=cv2.INTER_AREA)

    # Extraer los canales BGR y Alfa
    b, g, r, a = cv2.split(imagen_transparente)

    # Normalizar la máscara alfa
    mascara_alpha = a / 255.0

    # Recortar la región del fondo donde se colocará la imagen
    fondo_recorte = fondo[y_offset:y_offset+h_img, x_offset:x_offset+w_img]

    # Aplicar la imagen sin fondo sobre el fondo
    for c in range(3): # BGR
        fondo_recorte[:, :, c] = (fondo_recorte[:, :, c] * (1 - mascara_alpha) +
        imagen_transparente[:, :, c] * mascara_alpha)

    # Insertar la imagen combinada en el fondo original
    fondo[y_offset:y_offset+h_img, x_offset:x_offset+w_img] = fondo_recorte

# Guardar y mostrar el resultado
cv2.imwrite("imagen_final.png", fondo)
cv2.imshow("Imagen Final", fondo)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Ilustración 22 Código para sobreponer imágenes

Imágenes Utilizadas

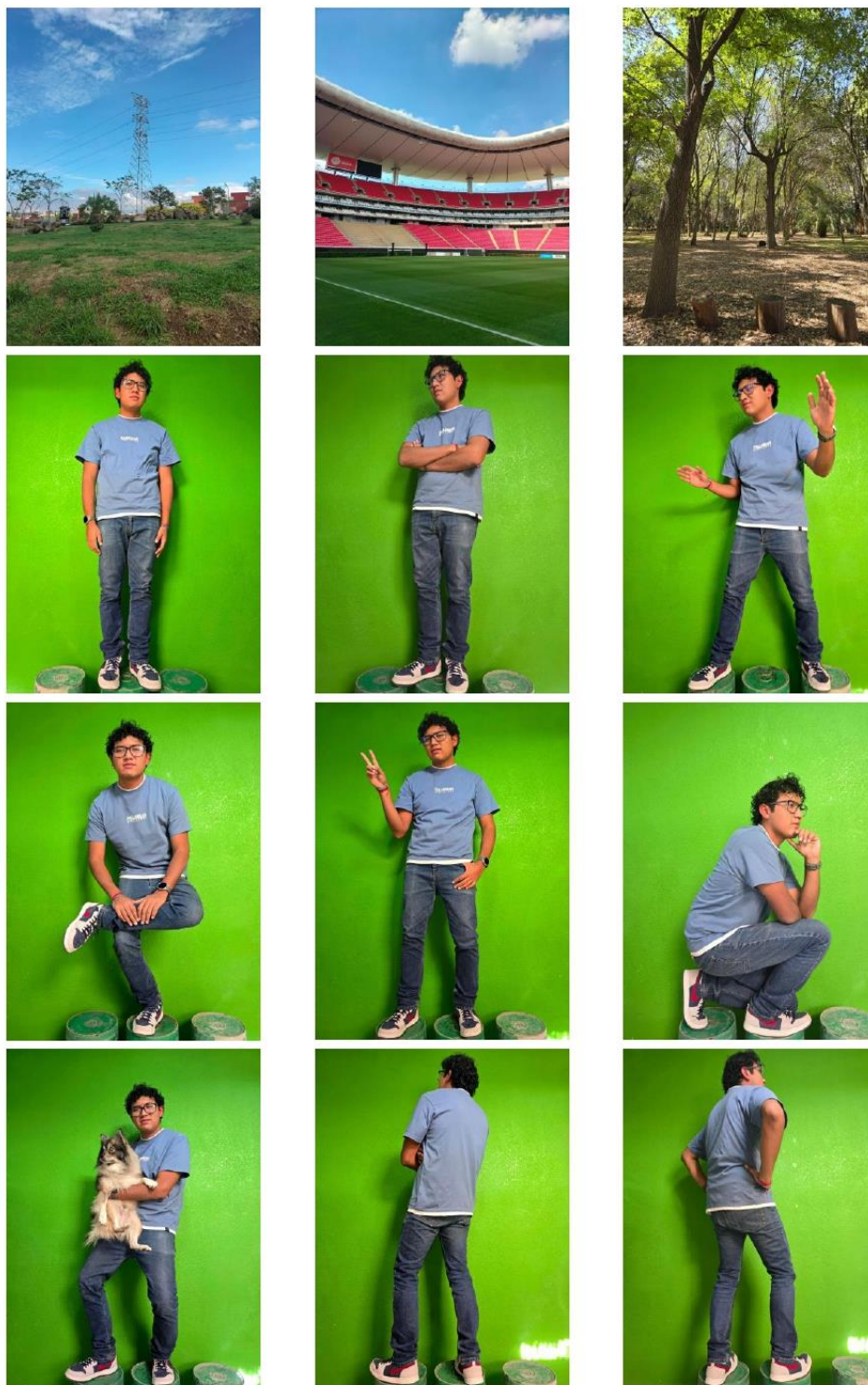


Ilustración 23 Fotos utilizadas

Resultados



Ilustración 24 Resultado de imágenes sin fondo

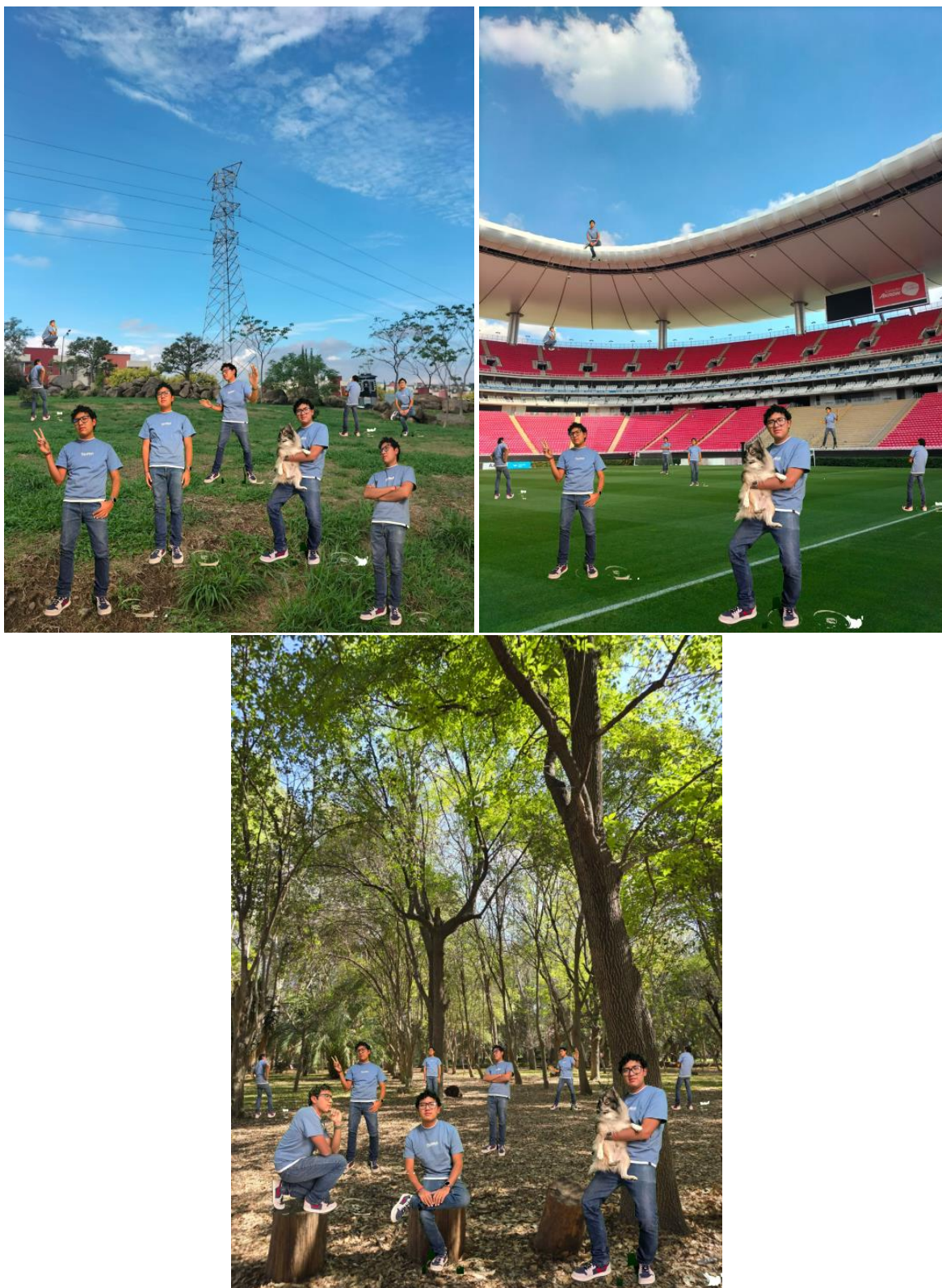


Ilustración 25 Resultados finales

Referencias

Espacios de Color (HSV):

https://docs.opencv.org/4.x/d9d/tutorial_py_colorspaces.html

Segmentación con cv2.inRange():

https://docs.opencv.org/4.x/da/d97/tutorial_threshold_inRange.html

Transparencia en OpenCV:

<https://pyimagesearch.com/2021/01/19/opencv-transparency-and-alpha-channel/>

Operaciones con cv2.bitwise_not():

https://docs.opencv.org/4.x/d0/d86/tutorial_py_image_arithmetics.html#autotoc_md1212

Redimensionamiento (cv2.resize()):

<https://learnopencv.com/image-resizing-with-opencv>

Separación de Canales (cv2.split()):

<https://stackoverflow.com/questions/19181485/splitting-image-using-opencv-in-python>

ChatGPT